# JS Concepts + DSA Patterns — WAMO Round 2

Muhammad Faizan Yousaf | Sunday April 26, 2026

## SECTION 1 — String Manipulation (my name is Faizan → Faizan is my name)

**Q: Reverse a string.**

➜ Split into array, reverse, join back.

```
function reverseStr(s) {
return s.split('').reverse().join('');
}
reverseStr('Faizan'); // 'nazi■F'
```

---

**Q: Reverse words in a sentence. ('my name is Faizan' → 'Faizan is name my')**

➜ Split by space, reverse the array of words, join back.

```
function reverseWords(s) {
return s.split(' ').reverse().join(' ');
}
reverseWords('my name is Faizan'); // 'Faizan is name my'
```

■ *Different from reversing characters. They test this to check if you read the problem.*

---

**Q: Rearrange sentence: 'my name is Faizan' → 'Faizan is my name' (move last word to front)**

➜ Split into words, pop last word, unshift to front.

```
function lastWordToFront(s) {
const words = s.split(' ');
const last = words.pop(); // remove last word
words.unshift(last); // add to front
return words.join(' ');
}
lastWordToFront('my name is Faizan'); // 'Faizan my name is'
// OR to get 'Faizan is my name' — sort by moving name to index 0:
// Use the reverseWords approach above
```

---

**Q: Check if a string is a palindrome. ('racecar' → true)**

➜ Two pointer from both ends. Clean string first.

```
function isPalindrome(s) {
s = s.toLowerCase().replace(/[^a-z0-9]/g, '');
let l = 0, r = s.length - 1;
while (l < r) {
if (s[l] !== s[r]) return false;
l++; r--;
}
return true;
}
```

---

**Q: Count character frequency in a string.**

➜ Use a hash map — key is character, value is count.

```
function charFreq(s) {
const map = {};
for (let c of s) {
map[c] = (map[c] || 0) + 1;
}
return map;
}
charFreq('faizan'); // {f:1, a:2, i:1, z:1, n:1}
```

---

**Q: Check if two strings are anagrams.**

➜ Sort both strings and compare. Or use frequency map.

```
function isAnagram(s, t) {
if (s.length !== t.length) return false;
return s.split('').sort().join('') === t.split('').sort().join('');
}
isAnagram('listen', 'silent'); // true
```

---

**Q: Find first non-repeating character.**

➜ Build frequency map, then find first character with count 1.

```
function firstUnique(s) {
const map = {};
for (let c of s) map[c] = (map[c] || 0) + 1;
for (let c of s) if (map[c] === 1) return c;
return null;
}
firstUnique('faizan'); // 'f'
```

---

**Q: Longest substring without repeating characters. (Sliding Window)**

➜ Use a set to track current window. Shrink from left when duplicate found.

```
function lengthOfLongestSubstring(s) {
const set = new Set();
let l = 0, max = 0;
for (let r = 0; r < s.length; r++) {
while (set.has(s[r])) {
set.delete(s[l]);
l++;
}
set.add(s[r]);
max = Math.max(max, r - l + 1);
}
return max;
}
lengthOfLongestSubstring('abcabcbb'); // 3 ('abc')
```

■ *Classic sliding window. State this pattern name out loud.*

---

## SECTION 2 — JavaScript Medium Concepts

**Q: What does this print and why? (Classic this trap)**

➜ Prints undefined. setTimeout's regular function loses 'this' context — it refers to global (window/undefined in strict). Fix: use arrow function.

```
const obj = {
name: 'Faizan',
greet: function() {
setTimeout(function() {
console.log(this.name); // undefined — 'this' is global
}, 100);
}
};
// Fix: arrow function inherits 'this' from greet
setTimeout(() => console.log(this.name), 100); // 'Faizan'
```

---

**Q: call vs apply vs bind — what's the difference?**

➜ All three set 'this' explicitly. call: invoke immediately with args one by one. apply: invoke immediately with args as array. bind: returns a NEW function with 'this' bound — doesn't invoke immediately.

```
function greet(greeting, punct) {
return `${greeting} ${this.name}${punct}`;
}
const user = { name: 'Faizan' };
greet.call(user, 'Hello', '!'); // 'Hello Faizan!' — immediate
greet.apply(user, ['Hello', '!']); // 'Hello Faizan!' — immediate, array args
const bound = greet.bind(user); // returns new function
bound('Hello', '!'); // 'Hello Faizan!' — call later
```

---

**Q: What is debounce? Write a simple implementation.**

➜ Debounce delays execution until after a pause in calls. Used for search input, resize events — prevents firing on every keystroke.

```
function debounce(fn, delay) {
let timer;
return function(...args) {
clearTimeout(timer);
timer = setTimeout(() => fn.apply(this, args), delay);
};
}
// Usage: fires 300ms after user stops typing
const search = debounce((query) => fetchResults(query), 300);
```

---

**Q: Debounce vs Throttle — difference?**

➜ Debounce: waits for pause, then fires ONCE. Throttle: fires at most ONCE per time interval regardless of how many calls.

```
// Throttle — fires every 300ms max
function throttle(fn, limit) {
let lastCall = 0;
return function(...args) {
const now = Date.now();
if (now - lastCall >= limit) {
lastCall = now;
fn.apply(this, args);
}
};
}
```

■ *Debounce = search bar. Throttle = scroll handler, resize.*

---

**Q: What is prototypal inheritance?**

➜ Every JS object has a prototype chain. When you access a property, JS looks up the chain until it finds it or reaches null. Classes in JS are syntactic sugar over this prototype system.

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function() { return this.name + ' speaks'; };
const dog = new Animal('Rex');
dog.speak(); // 'Rex speaks' — found on prototype
// ES6 class — same thing under the hood
class Animal {
constructor(name) { this.name = name; }
speak() { return this.name + ' speaks'; }
}
```

---

**Q: What causes memory leaks in JavaScript?**

➜ Global variables that are never cleared. Event listeners not removed when component unmounts. Closures holding large objects. Timers (setInterval) that keep running.

```
// Memory leak — interval never cleared
setInterval(() => doSomething(), 1000);
// Fix — store reference and clear
const id = setInterval(() => doSomething(), 1000);
clearInterval(id); // clear when done
// Memory leak — event listener not removed
element.addEventListener('click', handler);
// Fix:
element.removeEventListener('click', handler);
```

---

**Q: What is event delegation?**

➜ Instead of adding event listeners to each child element, add ONE listener to the parent. The event bubbles up and you check event.target to identify which child was clicked.

```
// Instead of listener on each
document.querySelector('ul').addEventListener('click', (e) => {
if (e.target.tagName === 'LI') {
console.log('Clicked:', e.target.textContent);
}
});
```

■ *Better performance. Works for dynamically added elements.*

---

**Q: What is the difference between null and undefined?**

➜ undefined: variable declared but no value assigned — JS sets it automatically. null: intentional empty value — YOU set it explicitly. typeof null is 'object' (historic JS bug).

```
let a; // undefined — not assigned
let b = null; // null — intentionally empty
typeof undefined // 'undefined'
typeof null // 'object' — JS quirk, not a mistake you made
null == undefined // true (loose equality)
null === undefined // false (strict equality)
```

---

**Q: Explain Promise chaining and error handling.**

➜ Each .then() returns a new Promise. .catch() at the end catches any error from any step in the chain. With async/await, use try/catch.

```
// Promise chain
fetch('/api/user')
.then(r => r.json())
.then(user => fetch(`/api/posts/${user.id}`))
.then(r => r.json())
.catch(err => console.error('Failed:', err));
// async/await equivalent
async function getData() {
try {
const user = await fetch('/api/user').then(r => r.json());
const posts = await fetch(`/api/posts/${user.id}`).then(r => r.json());
return posts;
} catch (err) {
console.error('Failed:', err);
}
}
```

## SECTION 3 — DSA Patterns with Code

### Pattern Overview

| Pattern | When to use | Key problems |
|---|---|---|
| Hash Map | Frequency count, lookup in O(1), pair finding | Two Sum, Group Anagrams, Subarray Sum |
| Sliding Window | Subarray/substring problems, contiguous elements | Longest substring, Max sum subarray |
| Two Pointers | Sorted array, pair finding, removing duplicates | 3Sum, Container with water, Palindrome |
| Stack | Matching brackets, next greater element, undo/redo | Valid Parenthesis, Daily Temperatures |
| Binary Search | Sorted array search, find boundary | Search rotated array, Find peak |
| BFS/DFS | Graph/tree traversal, connected components | Number of islands, Level order traversal |

**Q: Two Sum — indices of two numbers that add to target.**

➜ Hash map stores number→index. For each element check if complement exists. O(n).

```
function twoSum(nums, target) {
const map = {};
for (let i = 0; i < nums.length; i++) {
const diff = target - nums[i];
if (map[diff] !== undefined) return [map[diff], i];
map[nums[i]] = i;
}
}
twoSum([2,7,11,15], 9); // [0,1]
```

**Q: Group Anagrams — group strings that are anagrams of each other.**

➜ Sort each string as key in map. Group strings with same sorted key.

```
function groupAnagrams(strs) {
const map = {};
for (let s of strs) {
const key = s.split('').sort().join('');
if (!map[key]) map[key] = [];
map[key].push(s);
}
return Object.values(map);
}
groupAnagrams(['eat','tea','tan','ate','nat','bat']);
// [['eat','tea','ate'], ['tan','nat'], ['bat']]
```

**Q: Subarray Sum Equals K — count subarrays that sum to k.**

➜ Prefix sum + hash map. For each index, check if (currentSum - k) exists in map.

```
function subarraySum(nums, k) {
const map = {0: 1};
let sum = 0, count = 0;
for (let n of nums) {
sum += n;
if (map[sum - k]) count += map[sum - k];
map[sum] = (map[sum] || 0) + 1;
}
return count;
}
subarraySum([1,1,1], 2); // 2
```

■ *This is the hardest hash map pattern. If it comes up — prefix sum is the key idea.*

**Q: 3Sum — find all triplets that sum to zero.**

➜ Sort first. Fix one element, use two pointers for the rest. Skip duplicates.

```
function threeSum(nums) {
nums.sort((a,b) => a-b);
const result = [];
for (let i = 0; i < nums.length - 2; i++) {
if (i > 0 && nums[i] === nums[i-1]) continue; // skip dup
let l = i+1, r = nums.length-1;
while (l < r) {
const sum = nums[i] + nums[l] + nums[r];
if (sum === 0) {
result.push([nums[i], nums[l], nums[r]]);
while (nums[l] === nums[l+1]) l++;
while (nums[r] === nums[r-1]) r--;
l++; r--;
} else if (sum < 0) l++;
else r--;
}
}
return result;
}
```

---

**Q: Container With Most Water — two pointers.**

➜ Start from both ends. Move the pointer with the shorter height inward. Track max area.

```
function maxArea(height) {
let l = 0, r = height.length - 1, max = 0;
while (l < r) {
const area = Math.min(height[l], height[r]) * (r - l);
max = Math.max(max, area);
if (height[l] < height[r]) l++;
else r--;
}
return max;
}
```

---

**Q: Daily Temperatures — stack problem. Find days until warmer temperature.**

➜ Monotonic stack. Push indices. When current temp > stack top temp, pop and calculate days.

```
function dailyTemperatures(temps) {
const result = new Array(temps.length).fill(0);
const stack = []; // stores indices
for (let i = 0; i < temps.length; i++) {
while (stack.length && temps[i] > temps[stack[stack.length-1]]) {
const idx = stack.pop();
result[idx] = i - idx;
}
stack.push(i);
}
return result;
}
dailyTemperatures([73,74,75,71,69,72,76,73]);
// [1,1,4,2,1,1,0,0]
```

---

**Q: Search in Rotated Sorted Array — binary search.**

➜ Array was sorted then rotated. Find which half is sorted, then decide which half target is in.

```
function search(nums, target) {
let l = 0, r = nums.length - 1;
while (l <= r) {
const mid = Math.floor((l + r) / 2);
if (nums[mid] === target) return mid;
// left half is sorted
if (nums[l] <= nums[mid]) {
if (target >= nums[l] && target < nums[mid]) r = mid - 1;
else l = mid + 1;
} else { // right half is sorted
if (target > nums[mid] && target <= nums[r]) l = mid + 1;
else r = mid - 1;
}
}
return -1;
}
search([4,5,6,7,0,1,2], 0); // 4
```

---

**Q: 2nd Highest Number — confirmed Round 2 question.**

➜ Single pass, two variables. Handle duplicates. O(n) time, O(1) space.

```
function secondHighest(arr) {
let first = -Infinity, second = -Infinity;
for (let n of arr) {
if (n > first) { second = first; first = n; }
else if (n > second && n !== first) second = n;
}
return second === -Infinity ? null : second;
}
secondHighest([3,1,4,1,5,9,2,6]); // 6
secondHighest([5,5,5]); // null
```

■ *Always handle the duplicate case — they will test [5,5,5].*

---

## SECTION 4 — Problem Approach Framework (say this out loud)

| Step | What to say / do |
|---|---|
| 1. Read twice | Read problem. Re-read. Don't code anything yet. |
| 2. State I/O | "Input is an array of integers, output is two indices." |
| 3. Brute force first | "Brute force: nested loop, $O(n^2)$. Let me think if we can do better." |
| 4. Pattern recognition | "This looks like a hash map / sliding window / two pointer problem." |
| 5. Talk complexity | "This approach is O(n) time and O(n) space." |
| 6. Code simply | Write clean, named variables. No clever one-liners under pressure. |
| 7. Test with example | Walk through the example in the problem step by step. |
| 8. Edge cases | "What if array is empty? What if all duplicates? What if negative numbers?" |

*If you blank — say 'Let me think through brute force first.' Then start talking. Never go silent.*